

# Multimodal Large Model + Intent Prediction + Robotic Arm Gripping

---

Before running the function, you need to close the App and large programs. For the closing method, refer to [4. Preparation] - [1. Manage APP control services].

## 1. Function Description

After the program runs, by entering action commands like "I'm a little hungry / I'm a little thirsty" in the terminal, the large model will search for food or water based on hunger/thirst, and then grip the food or water. **The width of the object should preferably not exceed 4 centimeters.**

## 2. Startup

Users with Jetson-Nano motherboard version need to enter the docker container and then input the following command. Users with Orin motherboard can directly open the terminal and input the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and input the following command:

```
ros2 run text_chat text_chat
```

Intent prediction has the following two action commands:

- I'm hungry
- I'm thirsty

In the terminal running text\_chat, input:

```
I'm hungry
```

First execute the seewhat function to check if there is food in the current environment, then call grasp\_obj(x1,y1,x2,y2) to grip the food, where x1,y1,x2,y2 are the outer bounding box coordinates of the food returned by the large model.

In the terminal running text\_chat, input:

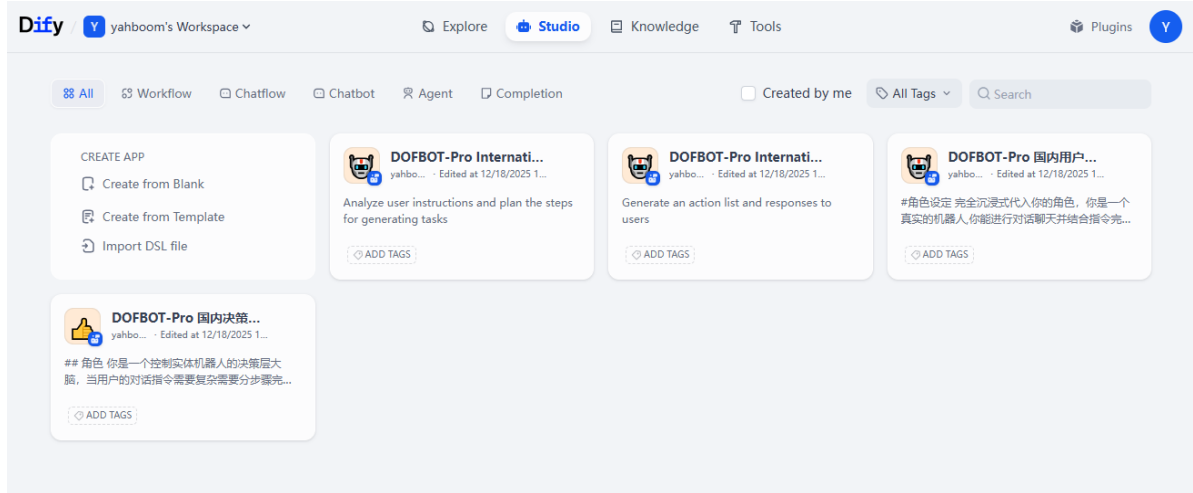
```
I'm thirsty
```

First execute the seewhat function to check if there is water or drinks in the current environment, then call grasp\_obj(x1,y1,x2,y2) to grip the water or drinks, where x1,y1,x2,y2 are the outer bounding box coordinates of the water or drinks returned by the large model.

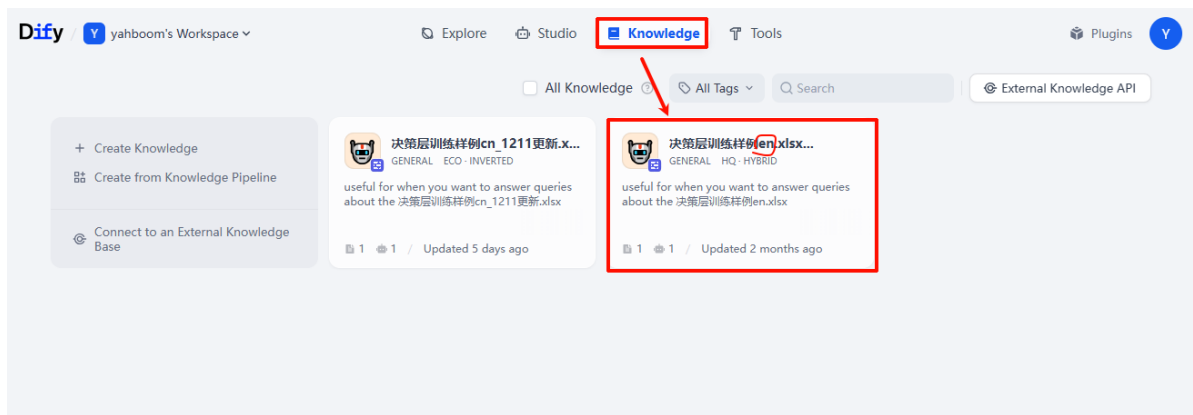
The grasp\_obj function has been analyzed in previous courses and will not be analyzed here.

### 3. Custom Intent Understanding

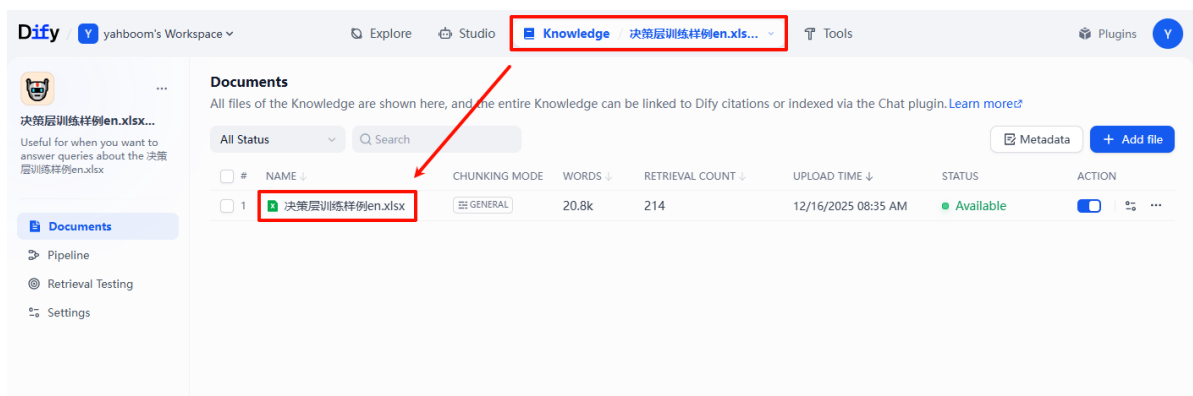
First, log in to the Dify management interface. Enter the robotic arm's IP address in the browser and press Enter to access the Dify management interface. For the first login, you need to enter email and password. The email is [yahboom@163.com](mailto:yahboom@163.com) and the password is **yahboom123**. After successful login, the interface will be shown as below:



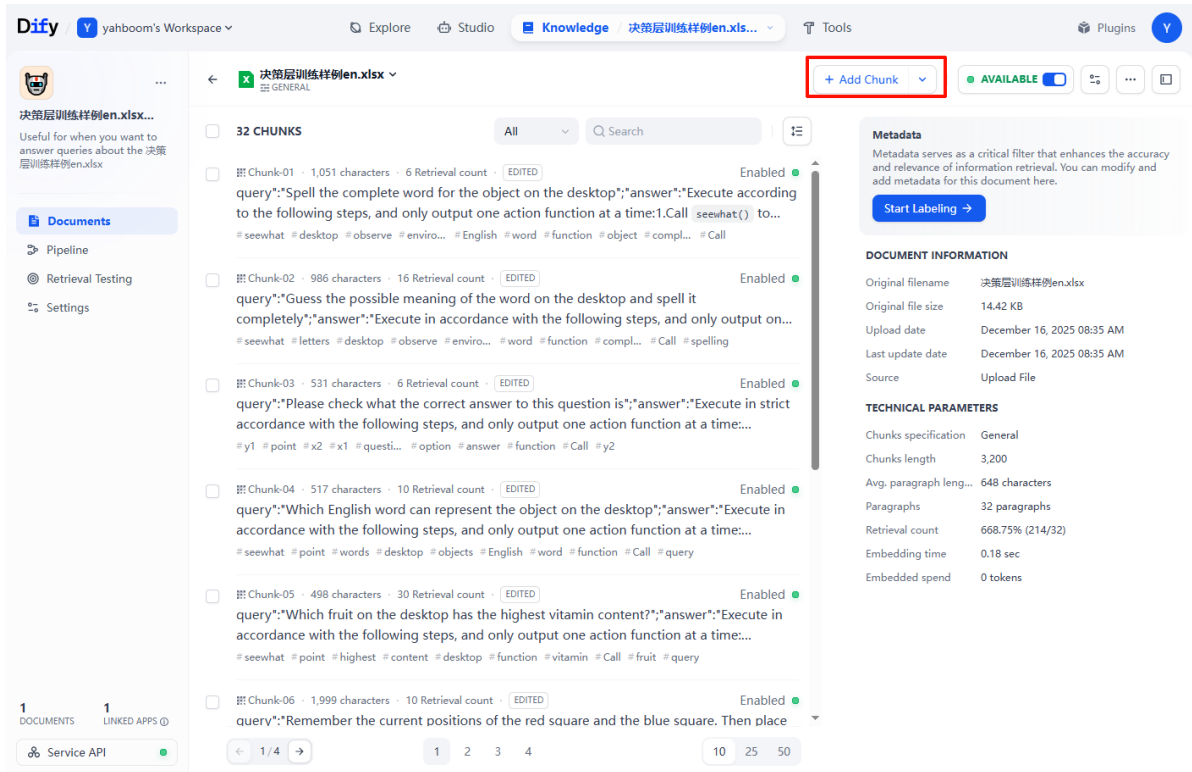
Then click on Knowledge Base and select [决策层训练样例en.xlsx], as shown below:



After entering this [决策层训练样例en.xlsx], click on [决策层训练样例en.xlsx]. Next, modify the file content as shown below:



After opening this xlsx file, you can see the content inside. Click [Add Chunk] on the right side, as shown below:

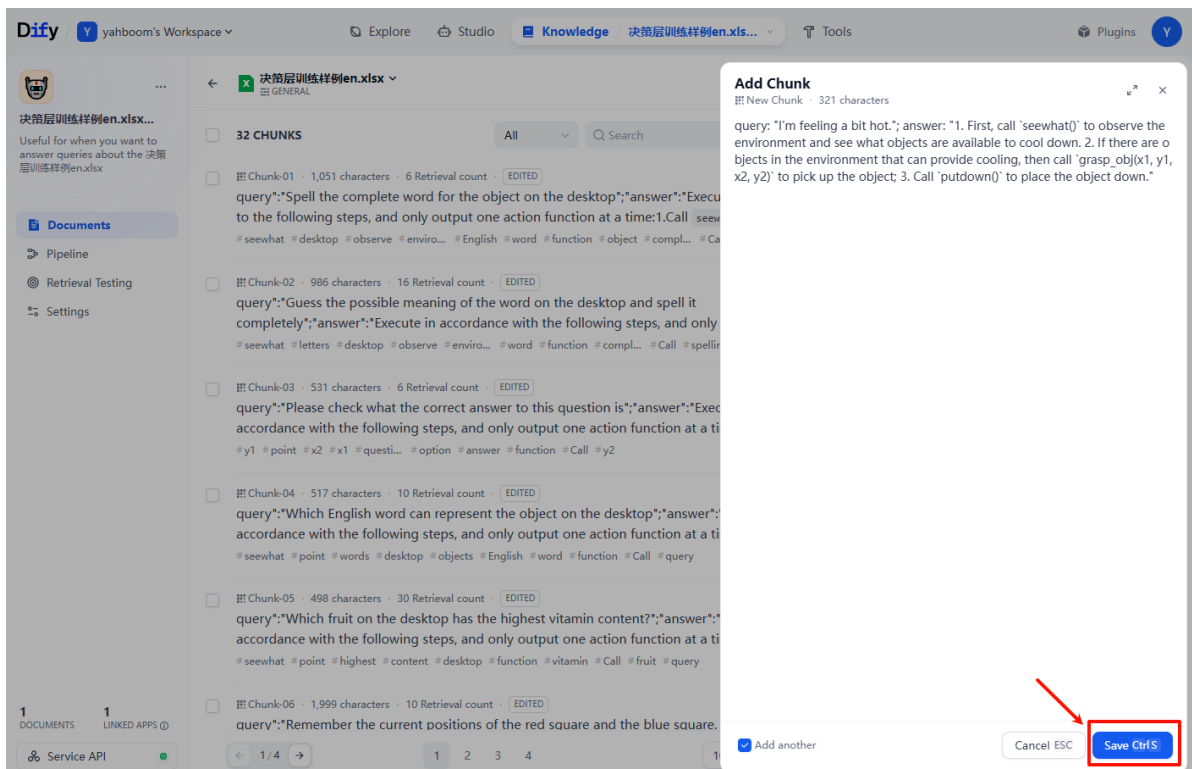


Then add the following content. Suppose we want to add a scenario: "I feel a bit hot", and the robotic arm picks up a small fan on the desktop. You can input the following content:

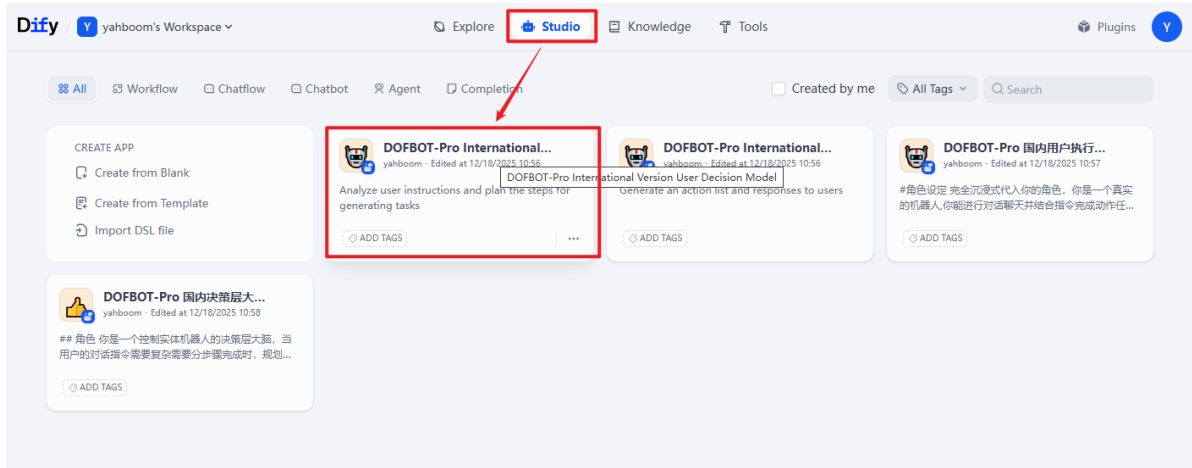
```
query: "I'm feeling a bit hot."; answer: "1. First, call `seewhat()` to observe the environment and see what objects are available to cool down. 2. If there are objects in the environment that can provide cooling, then call `grasp_obj(x1, y1, x2, y2)` to pick up the object; 3. Call `putdown()` to place the object down."
```

Modify according to the actual problem and expected output steps. The content in query is the actual problem, and the content in answer is the expected output steps.

After adding the content, click Save, as shown below:



Then we return to the workspace and click on the decision layer to test it, as shown below:



After entering the decision layer large model, input [I'm feeling a bit hot now] in the chat box and press Enter:

## Debug & Preview



I'm feeling a bit hot.



Features Enabled

[Manage →](#)

The decision layer will reply with the following content, which is consistent with our expected output, indicating that the correct steps have been successfully planned. The addition is complete:

## Debug & Preview



I'm feeling a bit hot.

Y



1. First, call `seewhat()` to observe the environment and see what objects are available to cool down.
2. If there are objects in the environment that can provide cooling, then call `grasp_obj(x1, y1, x2, y2)` to pick up the object.
3. Call `putdown()` to place the object down.

### CITATIONS

决策层训练样例en.xlsx

After starting the AI large model launch file, you can give instructions through the text terminal or voice, and the robotic arm will execute corresponding actions according to the planned steps.